

Эффективная реализация анонимных функций в объектно-ориентированных языках

Болохононов Артем Владимирович, гр. 24152
Научный руководитель: Трепаков Иван Сергеевич

Анонимные функции

В объектно-ориентированных языках программирования

В объектно-ориентированных языках лямбда-функции — это абстракция над синтетическими классами.

```
val capture = "arg = "  
  
val lambda = { x => println(capture + x) }  
lambda(42)
```

Анонимные функции

В объектно-ориентированных языках программирования

В объектно-ориентированных языках лямбда-функции — это абстракция над синтетическими классами.

```
val capture = "arg = "  
val lambda = new Lambda$$1(capture)  
lambda(42)  
  
class Lambda$$1(val prefix: Str) extends (Int ⇒ Unit) {  
  override def apply(x: Int) = {  
    println(prefix + x)  
  }  
}
```

Анонимные функции

В объектно-ориентированных языках программирования

В объектно-ориентированных языках лямбда-функции — это абстракция над синтетическими классами.

```
val capture = "arg = "  
val lambda = new Lambda$$1(capture)  
lambda(42)  
  
class Lambda$$1(val prefix: Str) extends (Int ⇒ Unit) {  
    override def apply(x: Int) = {  
        println(prefix + x)  
    }  
}
```

Таким образом, анонимные функции в объектно-ориентированных языках — это полноценный объект, который требует как аллокации, так и сборки мусора. Из-за чего возникают издержки по памяти и времени.

Постановка задачи

Цель: оптимизация издержек представления анонимных функций в объектно-ориентированных языках программирования.

Постановка задачи

Цель: оптимизация издержек представления анонимных функций в объектно-ориентированных языках программирования.

Задачи:

1. Изучить существующие подходы к уменьшению издержек на выделение объектов в куче,
2. Разработать подход к оптимизации издержек анонимных функций и функциональных замыканий,
3. Выполнить экспериментальную реализацию предложенного подхода,
4. Провести апробацию на представительном наборе тестов и приложений.

Оптимизации

Анализ утеканий

```
class SomeObject(var a: Int, var b: Double, c: String)

val obj = SomeObject(42, 42.0, "42")
useField(obj.a)
useField(obj.b)
obj.c = "override"

if (someBoolean) {
    method(obj)
}

if (anotherBoolean) {
    return obj
}
```

Оптимизации

Анализ утеканий

```
class SomeObject(var a: Int, var b: Double, c: String)

val obj = SomeObject(42, 42.0, "42")
useField(obj.a)
useField(obj.b)
obj.c = "override"
```


Оптимизации

Анализ утеканий

```
class SomeObject(var a: Int, var b: Double, c: String)

val (objA, objB, objC) = (42, 42.0, "42")
useField(objA)
useField(objB)
objC = "override"
```

Оптимизации

Анализ утеканий

```
class SomeObject(var a: Int, var b: Double, c: String)

val (objA, objB, objC) = (42, 42.0, "42")
useField(objA)
useField(objB)
objC = "override"
```

скаляризованное

Оптимизации

Анализ утеканий

```
class SomeObject(var a: Int, var b: Double, c: String)

val (objA, objB, objC) = (42, 42.0, "42")
useField(objA)
useField(objB)
objC = "override"

if (someBoolean) {
    return obj // ?
}
```

скаляризованное

Оптимизации

Анализ утеканий

```
class SomeObject(var a: Int, var b: Double, c: String)

val (objA, objB, objC) = (42, 42.0, "42")
useField(objA)
useField(objB)
objC = "override"

if (someBoolean) {
    return rematerialize(objA, objB, objC)
}
```

скаляризованное

Оптимизации

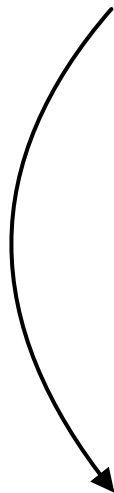
Анализ утеканий

```
class SomeObject(var a: Int, var b: Double, c: String)

val (objA, objB, objC) = (42, 42.0, "42")
useField(objA)
useField(objB)
objC = "override"

if (someBoolean) {
    return rematerialize(objA, objB, objC)
}
```

скаляризованное



в куче

Оптимизации

Анализ утеканий

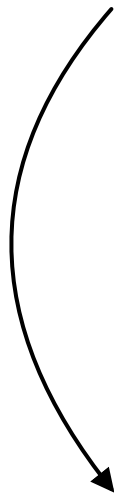
```
class SomeObject(var a: Int, var b: Double, c: String)

val (objA, objB, objC) = (42, 42.0, "42")
useField(objA)
useField(objB)
objC = "override"

if (someBoolean) {
    return rematerialize(objA, objB, objC)
}

if (anotherBoolean) {
    method(obj) // ?
}
```

скаляризованное



в куче

Оптимизации

Анализ утеканий

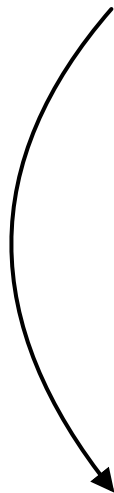
```
class SomeObject(var a: Int, var b: Double, c: String)

val (objA, objB, objC) = (42, 42.0, "42")
useField(objA)
useField(objB)
objC = "override"

if (someBoolean) {
    return rematerialize(objA, objB, objC)
}

if (anotherBoolean) {
    // метод не содержит утеканий
    method(reconstruct(obj)) // на стеке
}
```

скаляризованное



в куче

Оптимизации

Анализ утеканий

```
class SomeObject(var a: Int, var b: Double, c: String)

val (objA, objB, objC) = (42, 42.0, "42")
useField(objA)
useField(objB)
objC = "override"

if (someBoolean) {
    return rematerialize(objA, objB, objC)
}

if (anotherBoolean) {
    // метод не содержит утеканий
    method(reconstruct(obj)) // на стеке
}
```



Оптимизации

Анализ утеканий

```
class SomeObject(var a: Int, var b: Double, c: String)

val (objA, objB, objC) = (42, 42.0, "42")
useField(objA)
useField(objB)
objC = "override"

if (someBoolean) {
    return rematerialize(objA, objB, objC)
}

if (anotherBoolean) {
    // метод не содержит утеканий
    method(reconstruct(obj)) // на стеке
}

def method(o: SomeObject) = {
    if (anotherBoolean2) { // очень редко
        staticField = o
    }
}
```



Оптимизации

Анализ утеканий

```
class SomeObject(var a: Int, var b: Double, c: String)

val (objA, objB, objC) = (42, 42.0, "42")
useField(objA)
useField(objB)
objC = "override"

if (someBoolean) {
    return rematerialize(objA, objB, objC)
}

if (anotherBoolean) {
    // метод не содержит утеканий
    method(reconstruct(obj)) // на стеке --- некорректно
}

def method(o: SomeObject) = {
    if (anotherBoolean2) { // очень редко
        staticField = o
    }
}
```



Оптимизации

Анализ утеканий

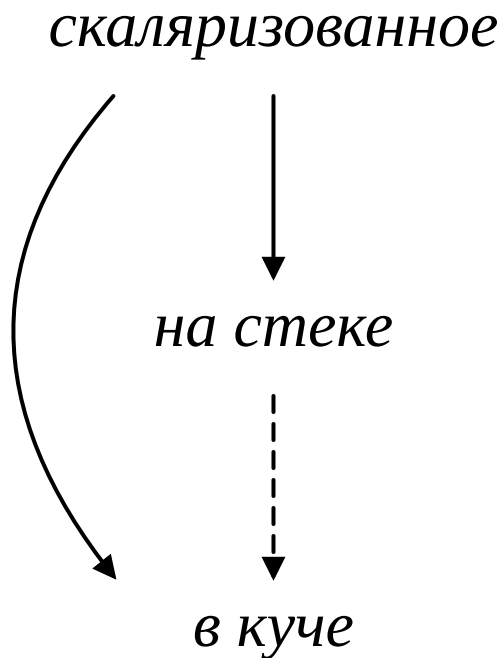
```
class SomeObject(var a: Int, var b: Double, c: String)

val (objA, objB, objC) = (42, 42.0, "42")
useField(objA)
useField(objB)
objC = "override"

if (someBoolean) {
    return rematerialize(objA, objB, objC)
}

if (anotherBoolean) {
    // метод не содержит утеканий
    method(reconstruct(obj)) // на стеке --- некорректно
}

def method(o: SomeObject) = {
    if (anotherBoolean2) { // очень редко
        staticField = o
    }
}
```



Межпроцедурный анализ частичных утеканий

Идея

```
val lambda = { x => println(x) }  
lambdaUse(lambda)  
  
def lambdaUse(x: (Int => Unit)) = {  
  ...  
  if (globalFlag) { // Часто false  
    globalVar = x  
  }  
  ...  
}
```

Межпроцедурный анализ частичных утеканий

Идея

```
val lambda = onStack( { x => println(x) } )
lambdaUse(lambda)

def lambdaUse(x: (Int => Unit)) = {
  ...
  if (globalFlag) { // Часто false
    globalVar = evacuate(x)
  }
  ...
}
```

Межпроцедурный анализ частичных утеканий

Идея

```
val lambda = onStack( { x => println(x) } )
lambdaUse(lambda)

def lambdaUse(x: (Int => Unit)) = {
  ...
  if (globalFlag) { // Часто false
    globalVar = evacuate(x)
  }
  ...
}
```

- Будем перемещать объект на стек, если у него нет гарантированного *утекающего* использования,
- Если такие использования есть, то перед их исполнением будем вставлять операцию *эвакуация*, которая создаст копию объекта на куче.

Межпроцедурный анализ частичных утеканий

Алгоритм

Данный анализ является расширением анализа утеканий и является потоковым анализом.

Межпроцедурная часть

```
call[SomeMethod](param1, param2, param3, ... )
```

The diagram illustrates the escape analysis for the parameters of the `call[SomeMethod]` function. Each parameter (`param1`, `param2`, and `param3`) is associated with a vertical line. Below each line is a box divided into two sections: `NoEscape` on top and `Escape` on the bottom. This represents the state of the escape analysis for each parameter.

Межпроцедурный анализ частичных утеканий

Алгоритм

Данный анализ является расширением анализа утеканий и является потоковым анализом.

Межпроцедурная часть

`call[SomeMethod](param1, param2, param3, ... )`

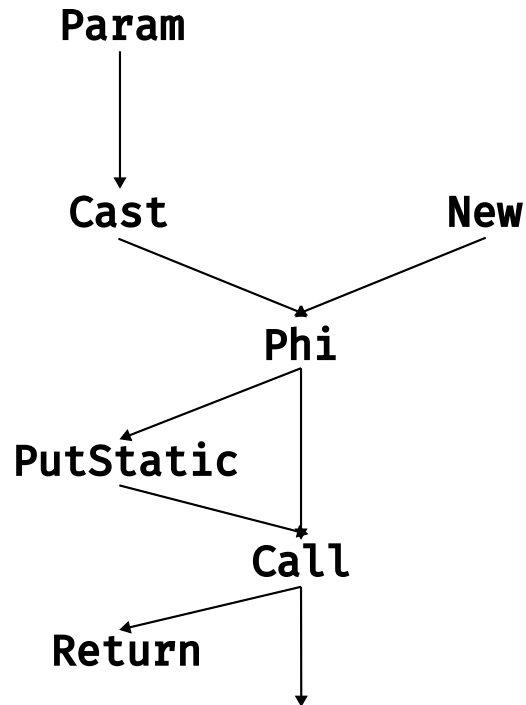
Diagram illustrating the escape analysis for parameters `param1`, `param2`, and `param3`. Each parameter is associated with a box containing the labels `NoEscape` and `Escape`, indicating the analysis results for each parameter.

`call[SomeMethod](param1, param2, param3, ... )`

Diagram illustrating the potential escape analysis for parameters `param1`, `param2`, and `param3`. Each parameter is associated with a box containing the labels `NoEscape`, `Potential`, and `Guaranteed`, indicating the analysis results for each parameter.

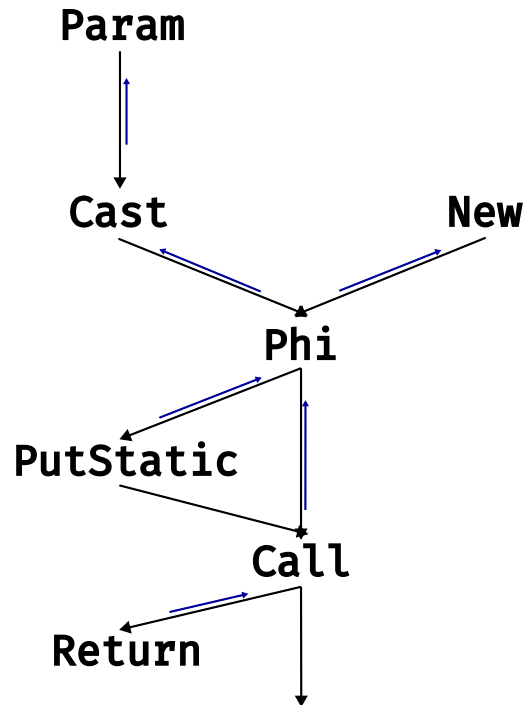
Межпроцедурный анализ частичных утеканий

Локальная часть



Межпроцедурный анализ частичных утеканий

Локальная часть



Межпроцедурный анализ частичных утеканий

Классификация использований

Утеканиями будем считать:

1. Возврат из метода;
2. Запись анализируемого объекта:
 - В поле другого объекта,
 - В качестве элемента массива,
 - В статическую переменную;

Межпроцедурный анализ частичных утеканий

Классификация использований

Утеканиями будем считать:

1. Возврат из метода;
2. Запись анализируемого объекта:
 - В поле другого объекта,
 - В качестве элемента массива,
 - В статическую переменную;
3. *Вызов функции, если для соответствующего параметра вычислилось, что он утекает.*

Межпроцедурный анализ частичных утеканий

Оптимизации

1. Быстрые пути в эвакуации

ALLOCATE_AND_COPY

Межпроцедурный анализ частичных утеканий

Оптимизации

1. Быстрые пути в эвакуации

```
    cmp    object, 0          ; если объект = null
    jne    else
    mov    ecx, 0            ; возвращаем null
    jmp    continue
else:
    mov    eax, [object]
    test   eax, eax          ; если объект уже на куче
    jne    else_if
    mov    ecx, object       ; возвращаем его же
    jmp    continue
else_if:
    ALLOCATE_AND_COPY        ; иначе копируем
continue:
    ...
```

Межпроцедурный анализ частичных эвакуаций

Оптимизации

2. Дополнительный стековый слот для сохранения результата

```
cmp    object, 0      ; если объект = null
jne    else
mov     ecx, 0         ; возвращаем null
jmp     continue
else:
mov     eax, [object]
test    eax, eax       ; если объект уже на куче
jne     else_if
mov     ecx, object    ; возвращаем его же
jmp     continue
else_if:
        ALLOCATE_AND_COPY ; иначе копируем
continue:
        ...
```

```
→  HEADER
    FIELD_1
    FIELD_2
    FIELD_3
    ...
```

Межпроцедурный анализ частичных эвакуаций

Оптимизации

2. Дополнительный стековый слот для сохранения результата

```
cmp    object, 0      ; если объект = null
jne    else
mov     ecx, 0         ; возвращаем null
jmp     continue
else:
mov     eax, [object]
test    eax, eax       ; если объект уже на куче
jne     else_if
mov     ecx, object    ; возвращаем его же
jmp     continue
else_if:
    ALLOCATE_AND_COPY  ; иначе копируем
continue:
    ...
```

```
    ALLOCATED_SPACE (= nullptr)
→   HEADER
    FIELD_1
    FIELD_2
    FIELD_3
    ...
```

Межпроцедурный анализ частичных эвакуаций

Оптимизации

2. Дополнительный стековый слот для сохранения результата

```
    cmp    object, 0          ; если объект = null
    jne    else
    mov    ecx, 0             ; возвращаем null
    jmp    continue
else:
    mov    eax, [object]
    test   eax, eax           ; если объект уже на куче
    jne    get_from_cache
    mov    ecx, object        ; возвращаем его же
    jmp    continue
get_from_cache:
    mov    eax, [object-8]    ; получаем значение из кэша
    test   eax, eax           ; проверяем наличие объекта
    jne    else_if
    mov    ecx, eax           ; возвращаем из кэша
    jmp    continue
else_if:
    ALLOCATE_AND_COPY        ; иначе копируем
continue:
    ...
```

```
    ALLOCATED_SPACE (= nullptr)
→  HEADER
    FIELD_1
    FIELD_2
    FIELD_3
    ...
```

```
def evacuation[T](x: T): T = {
  val result = ...
  (&x - 8) = result
  return result
}
```

Окружение

Многоязыковая виртуальная машина Huawei

1. Оптимизирующий статический компилятор,
2. Управляемая среда исполнения,
3. Поддержка языков Java, Scala, Cangjie.



Результаты

Количество перемещений на стек; больше — лучше

Название теста	Анализ утеканий	Межпроцедурный анализ частичных утеканий
Статический компилятор	394	4488

Количество выделений объектов помеченных типов; меньше — лучше

Название теста	Анализ утеканий	Межпроцедурный анализ частичных утеканий
Компиляция Hello World	5'058'176	3'717'185
Компиляция scala.lisp	8'033'662	4'992'872
Компиляция scala.life	14'964'694	8'219'137

Заключение

В ходе работы было сделано:

1. Изучены существующие подходы к уменьшению издержек на выделение объектов в куче,
2. Разработано и реализовано расширение анализа утеканий «межпроцедурный анализ частичных утеканий»,
3. Полученное решение было протестировано.

Направления дальнейшей работы

1. Расширить анализ на другие типы, помимо лямбда-функций,
2. Добавить поддержку использования профилировочной информации.

Спасибо за внимание

Анализ утеканий

Алгоритм

NoEscape



RetEscape

RcvEscape



RetRcvEscape



GlobalEscape

Межпроцедурный анализ частичных эвакуаций

Алгоритм

